University of Engineering and Technology, Lahore

Mobile_Robotics_Lab_4

Lab_Report

Submitted by: 2022_MC_27

1. Objectives

This laboratory session covered three foundational ROS 2 tools: the launch system, rosbag2 data recording, and rqt visualization. Specific goals were:

- Implement ROS 2 launch files using LaunchDescription, Node, and ExecuteProcess actions.
- Record and verify live topic data using rosbag2 in SQLite3 format.
- Visualize topic data in real time using rqt_graph and rqt_plot.
- Implement a delayed waypoint-tracking controller enabling turtle2 to retrace turtle1's path.
- Analyze extracted trajectory data to evaluate follower performance.

2. Introduction

2.1 ROS 2 Launch System

A ROS 2 launch file returns a LaunchDescription object populated with actions such as Node (to start a persistent ROS 2 node) and ExecuteProcess (to run a one-shot shell command). This lab exercises three layers of complexity: a minimal two-node launcher, a three-action launcher that spawns a second turtle via a service call, and a four-action launcher that additionally starts a custom Python control node.

2.2 Rosbag2 and Data Recording

Rosbag2 records topic messages to disk in CDR (Common Data Representation) binary format. The default storage backend is SQLite3. Each recording produces a .db3 database file and a metadata.yaml that summarises duration, message counts, and QoS profiles. Offline analysis should be performed using rosbag2_py or subscriber-based deserialization rather than manual struct unpacking, as the latter is fragile for complex message types.

2.3 rqt Visualization Tools

rqt_graph renders the live publish-subscribe graph, confirming node-topic connectivity. rqt_plot streams numeric field values from any topic as time-series charts, enabling real-time observation of velocity command profiles.

2.4 Delayed Waypoint-Tracking Controller

Classical Follow-the-Leader (FTL) causes a follower to retrace the leader's path rather than directly pursuing its current position. In this implementation, the leader's pose is appended to a FIFO buffer of up to 50 waypoints. The follower targets the oldest buffered waypoint and removes it upon arrival within 0.2 units, introducing deliberate lag so turtle2 retraces turtle1's historical path rather than converging on its instantaneous position.

3. Methodology

3.1 Launch Files

Three launch files were developed incrementally:

- turtlesim_launch.py — Baseline: starts turtlesim_node and turtle_teleop_key.
- task_1_launch.py — Extends the baseline by adding an ExecuteProcess action that calls the /spawn service to place turtle2 at (5.0, 5.0, theta = 0.0). ExecuteProcess is used because the spawn call is a one-shot service invocation with no persistent node. The turtle is controlled using teleop (keyboard input) without any modifications to the control mechanism.
- task_2_launch.py — Adds a Node action launching the follow_leader node from my_launch_pkg, completing the four-component system.

3.2 Rosbag Recording

Recording was started after launching task_2_launch.py:

```
ros2 bag record /turtle1/pose /turtle2/pose /turtle1/cmd_vel /turtle2/cmd_vel
```

The session ran for 191.856 seconds. The resulting metadata.yaml was inspected to confirm message counts and recording integrity.

3.3 Follow-the-Leader Node

The follower node (follow_the_leader.py) subscribes to /turtle1/pose and /turtle2/pose, stores the leader's trajectory as a FIFO waypoint buffer (maximum 50 entries), and publishes Twist commands to /turtle2/cmd_vel at 10 Hz. An activation guard prevents the control loop from running until at least 10 waypoints are buffered, avoiding erratic motion during the initial fill phase. The proportional control law is:

```
cmd.linear.x  = 2.0 * distance
cmd.angular.z = 6.0 * angle_error
```

This is a proportional unicycle controller commonly used for waypoint tracking. Angle error is normalized to [-pi, pi] using atan2 before applying the gain.

4. Findings

4.1 Rosbag Metadata

Table 4.1: Rosbag Metadata Summary

Property	Value
Bag Format	SQLite3 (rosbag2)
Recording Duration	191.856 seconds (~3.2 min)
Total Messages	24,361
Serialization	CDR (Common Data Representation)
Storage Backend	sqlite3
Compression	None

4.2 Recorded Topics

Table 4.2: Recorded Topics

Topic	Message Type	Count	Purpose
/turtle1/pose	turtlesim/msg/Pose	11,992	Leader position & heading
/turtle2/pose	turtlesim/msg/Pose	11,992	Follower position & heading
/turtle1/cmd_vel	geometry_msgs/msg/Twist	246	Teleop commands to leader
/turtle2/cmd_vel	geometry_msgs/msg/Twist	131	Computed commands to follower

Both pose topics received equal message counts (11,992 each), consistent with turtlesim publishing both at the same rate. The cmd_vel disparity (246 vs 131) reflects the follower activating later and ceasing commands once its waypoint buffer is exhausted.

4.3 Trajectory Summary

Table 4.3: Trajectory Comparison — Leader vs Follower

Metric	Turtle1 (Leader)	Turtle2 (Follower)
Start Position	(5.54, 5.54)	(5.00, 5.00)
End Position	(8.41, 3.08)	(3.03, 8.36)
Total Path Length (units)	66.40	50.75
X Range (units)	0.69 – 10.49	0.61 – 9.88
Y Range (units)	1.23 – 10.43	1.16 – 10.19
Rows Stationary	87,912 / 108,655 (80.9%)	98,818 / 107,842 (91.6%)
Activation Delay	N/A	First move at row ~13,010

5. Extracted Trajectory Data

The full trajectory datasets (turtle1.csv and turtle2.csv, ~108,000 rows each) were extracted from the rosbag SQLite3 database. Tables 5.1 and 5.2 present sampled data at evenly spaced row intervals. Note that x, y, and theta are the only fields present in the CSV exports; the Pose message also carries linear_velocity and angular_velocity fields that reflect the simulator's estimated motion state.

5.1 Turtle1 (Leader) Sampled Trajectory

Table 5.1: Turtle1 — Sampled Pose (evenly spaced row intervals)

Sample Row	X	Y	Theta (rad)
0	5.544	5.544	0.000
10,865	5.262	4.374	-0.341
21,731	7.657	4.451	1.622
32,596	8.407	3.076	-1.962
43,462	8.407	3.076	-1.962
54,327	8.407	3.076	-1.962
65,193	8.407	3.076	-1.962
86,924	8.407	3.076	-1.962
97,789	8.407	3.076	-1.962
108,654	8.407	3.076	0.000

5.2 Turtle2 (Follower) Sampled Trajectory

Table 5.2: Turtle2 — Sampled Pose (evenly spaced row intervals)

Sample Row	X	Y	Theta (rad)
0	5.000	5.000	0.000
10,784	5.000	5.000	0.000
13,010	5.032	5.000	0.000
21,568	3.205	8.350	-2.506
32,352	3.025	8.356	-2.442
43,136	3.025	8.356	-2.442
64,705	3.025	8.356	-2.442
86,273	3.025	8.356	-2.442
97,057	3.025	8.356	-2.442
107,841	3.025	8.356	-2.442

5.3 Brief Trajectory Analysis

Turtle1 (leader) started at (5.54, 5.54) and covered a path length of 66.40 units across nearly the full simulation canvas (X: 0.69–10.49, Y: 1.23–10.43). However, 80.9% of its recorded rows show no positional change, reflecting extended idle periods during the session — visible as repeated identical entries in Table 5.1 from row 32,596 onward. The final resting position was (8.41, 3.08).

Turtle2 (follower) remained stationary at (5.00, 5.00) until row 13,010, when the waypoint buffer crossed the 10-entry activation threshold and the first small displacement appeared. Once active, it traversed 50.75 units — approximately 76% of the leader's total path. Its spatial range (X: 0.61–9.88, Y: 1.16–10.19) closely mirrors the leader's, confirming the FIFO strategy successfully reproduced the spatial extent of the explored region.

Turtle2 reached its final position (3.03, 8.36) before the end of the recording and remained there for the remainder of the session. This early stop occurred because the follower's proportional controller consumed waypoints faster than the leader was generating new ones once the leader became idle — the waypoint buffer emptied and no new entries were added.

One notable observation: 91.6% of turtle2's rows are stationary versus 80.9% for turtle1. This is expected given the follower's later activation and early buffer exhaustion. The disparity between the turtles' end positions ((8.41, 3.08) vs (3.03, 8.36)) is also expected — the follower was retracing historical waypoints with a lag, so it ended at a past position of the leader, not the leader's final position.

6. Observations and Discussion

6.1 Path-History Approach

The FIFO waypoint buffer successfully caused turtle2 to retrace turtle1's historical trajectory with a deliberate lag, achieving the desired path-following rather than direct pursuit behaviour. At any given instant, the follower operated in a different region of the simulation from the leader.

A known limitation is that the oldest buffered waypoint may require a sharp heading change, causing high angular velocity and briefly suppressed linear motion — a typical symptom of waypoint-chasing architectures with large angular gains.

6.2 Proportional Controller Limitations

The controller lacks velocity saturation. Because linear velocity scales as 2.0 * distance, large distances at buffer activation cause high-speed bursts that can overshoot waypoints. A practical improvement would be adding a linear velocity clamp (e.g., 1.5 units/s maximum) and an angular dead-zone to suppress micro-corrections when the heading error is already small.

6.3 Follower Activation Delay

The activation guard requires 10 buffered waypoints before the control loop runs. Because the leader was idle for extended periods (particularly the long pause visible in Table 5.1 from approximately rows 32,000–97,000), the buffer filled slowly and the follower's effective active window was short. For real-world deployment, a time-based lag or message-timestamp offset would be more predictable than a buffer-depth threshold.

7. Conclusion

This lab successfully demonstrated the integration of core ROS 2 tools through a structured workflow. The *turtlesim_launch.py* established the baseline system by launching the simulator, while *task_1_launch.py* extended functionality by spawning a second turtle using a one-time service call via ExecuteProcess, with control maintained through standard teleoperation.

The complete system, including rosbag2 and rqt tools, enabled reliable data recording, system validation, and visualization of robot behavior. The delayed waypoint-tracking controller allowed turtle2 to retrace turtle1's historical path, achieving approximately 76% path coverage. The observed performance limitations were mainly due to activation delay and buffer constraints, suggesting improvements such as velocity limiting, time-based lag, and smoother waypoint transitions.

8. Appendix: Source Code

A. turtlesim_launch.py — Baseline Launcher

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(package='turtlesim', executable='turtlesim_node', name='sim'),
        Node(package='turtlesim', executable='turtle_teleop_key',
name='teleop')
    ])
```

B. task_1_launch.py — Spawn turtle2

Extends baseline with an ExecuteProcess action to call the /spawn service:

```
ExecuteProcess(
    cmd=['ros2', 'service', 'call', '/spawn',
         'turtlesim/srv/Spawn',
         "{x: 5.0, y: 5.0, theta: 0.0, name: 'turtle2'}"],
    output='screen'
)
```

C. task_2_launch.py — Full Orchestration

Adds the custom follower node to complete the four-component system:

```
Node(package='my_launch_pkg', executable='follow_leader', name='follower')
```

D. Follow-the-Leader Control Loop

```
target_x, target_y = self.path[0]
dx = target_x - self.follower_pose.x
dy = target_y - self.follower_pose.y
distance    = math.sqrt(dx**2 + dy**2)
angle_error = math.atan2(dy, dx) - self.follower_pose.theta
angle_error = math.atan2(math.sin(angle_error), math.cos(angle_error))
cmd.linear.x  = 2.0 * distance
cmd.angular.z = 6.0 * angle_error
if distance < 0.2:
    self.path.pop(0)
```